

Experiment 18 — State estimation: the packing threshold is a floor under numeric linear algebra

Mathilda — execution-speed experiments

Contents

Experiment 18 — State estimation: the packing threshold is a floor under numeric linear algebra	1
Hypothesis	1
The kernels	1
Results	2
The finding: below the threshold there is no numeric linear algebra at all	2
What is still open, and why it was not done here	3
Why Mathilda is not the fastest here, and what it would take	3
Verification	4

Experiment 18 — State estimation: the packing threshold is a floor under numeric linear algebra

Date: 2026-07-31 · Code: `src/linalg/util.c`, `src/linalg/inv.c` · Result: Inverse of a Real matrix stopped returning exact Integer zeros — and a named, measured reason why a 6-state Kalman filter is 21× behind NumPy

Common method in [README.md](#).

Hypothesis

Every other benchmark in the five sweeps is dominated by the cost of arrays. This one is dominated by the cost of a call.

A constant-acceleration Kalman filter over 20000 measurements does, per step, two 6×6 matrix products, a 6×6 by 6 matrix–vector product, a 2×6 by 6×6, a 2×2 inverse and half a dozen small adds. The flop count for the whole run is about 2×10^7 — a single `dgemm` in the linear-algebra section does more than that in one call. What is being measured is 20000 iterations × ~10 operations of per-operation dispatch on data too small for any kernel to matter.

NumPy is not fast here either, and that is the point: the row shows where all three systems live when the data is small. An ensemble Kalman filter — the same estimator written the array way, 4096 members at once — sits beside it so the two regimes can be read together.

The kernels

id	kernel	what it stresses
kalman	6-state CA filter, 20000 steps	small dense matrices in a long loop
enkf	ensemble filter, 4096 members, 200 steps	the same estimator, array-shaped

Both are fully deterministic — the measurement sequence is a fixed pair of sinusoids — so $\text{Tr}[P]$ at the end is an exact cross-system check, and the filter is stable so it converges rather than drifting.

Results

Benchmark	Mathilda	Mathematica 14.0	NumPy 2.4.4	vs WL	vs NumPy
Kalman filter, 6 states, 20000 steps	9.16 s	824 ms	435 ms	1/11.1×	1/21.0×
Ensemble Kalman, 4096 members, 200 steps	3.77 s	2.96 s	92.7 ms	1/1.27×	1/40.6×

Both rows are essentially unchanged by this sweep, and both "regressions" in the before/after table (0.96×) are measurement variance — a controlled A/B on the same host has the fixed binary marginally ahead:

	baseline binary	fixed binary
$m \cdot v$, 6×6 by 6, ×50000	655 ms	603 ms
Inverse[2×2], ×20000	568 ms	536 ms

See [MOLECULAR_DYNAMICS.md](#) for the same caveat stated in full.

The finding: below the threshold there is no numeric linear algebra at all

PACK_MIN_ELEMENTS is 250, and it is 250 for good blast-radius reasons. A 6×6 matrix is 36 elements, so it is — correctly — never packed.

But builtin_inverse dispatches on linalg_call_has_ndarray. No NDArray, no ndla_inverse, no LAPACK. A 6×6 Real matrix therefore runs inverse_divfree: a fraction-free Gauss-Jordan elimination on the augmented matrix, symbolic, with an Expr allocation and an evaluate call per cell per pivot. Measured:

	per call
Inverse of a 2×2 Real matrix	26.8 μs
Inverse of a 6×6 Real matrix	~800 μs
$m \cdot v$, 6×6 by 6	12.1 μs
a 6×6 dgetrf+dgetri would be	~1 μs

The packing threshold is not only a performance heuristic. It is the boundary below which the numeric linear-algebra paths are unreachable.

That was not a designed decision; it fell out of dispatching on "is this already a buffer" rather than on "is this a matrix of machine numbers". Above 250 elements Inverse is LAPACK; below it, it is a computer-algebra algorithm being asked to do numerical work.

The bug this exposed

Running the symbolic elimination on a machine-real matrix also produced the wrong heads:

```
In[1]:= Inverse[{{2., 0.}, {0., 4.}}]
Out[1]= {{0.5, 0}, {0, 0.25}}      (* exact Integer zeros *)
```

Mathematica gives $\{\{0.5, 0.\}, \{0., 0.25\}\}$. The source was one line in `exact_div_wrapper`, the shared helper behind `Inverse`, `RowReduce` and `LinearSolve`:

```
if (is_zero_poly(num)) return expr_new_integer(0);
```

A zero numerator divides to zero — but at the numerator's own exactness. $0./5$ is $0.$, not 0 . Fixed, so a zero now carries the numerator's head, and the augmented identity block and every eliminated cell in `inverse_divfree` follow the input's exactness the same way `inverse_onestep` already did.

The visible half of that is the printed heads. The other half is that a matrix of mixed exact and inexact entries cannot be packed, so at any size above the threshold every operation downstream of an inverse with a structural zero in it fell off the buffer path.

What is still open, and why it was not done here

Routing small machine matrices to the numeric kernels. The obvious fix — in `builtin_inverse`, `pack_force` a matrix of machine reals regardless of size and call `ndla_inverse` — has a trap that is worth recording, because it is not visible until you write it:

`ndla_inverse` declines by calling `linalg_delist_and_reeval`, which rebuilds the call with plain Lists and re-evaluates it. That re-enters `builtin_inverse` with a plain List, which would pack it again, which would decline again — an infinite loop, entered only on the singular/degenerate inputs that are hardest to test. Doing this properly means splitting `ndla_inverse` into a `_try` form that returns NULL instead of degrading, and doing the same at `Dot`, `LinearSolve` and `Det`. That is a coherent piece of work and it is a phase, not a patch.

The **enkf** row at 40× NumPy is a different problem: `Transpose[an]` on a 4096×6 matrix, `Table[mn, {4096}]` to replicate a mean vector, and a 2×2 `Inverse` per step. The first is plan item 9.1 (transposed view), the second is a broadcast that has to allocate 4096 copies of a 6-vector because `Plus` threads the outer level, and the third is this experiment's finding.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
Kalman filter, 20000 steps	9.16 s	435 ms (NumPy)	21.0×	every matrix is below the packing threshold, so none of the operations are fast enough to be worth the overhead of a copying Transpose, a materialised broadcast, and
Ensemble Kalman, 4096 × 6	3.77 s	92.7 ms (NumPy)	40.6×	

The cause is one thing, and `state_estimation.m` measures it directly:

one operation	Mathilda	NumPy	ratio
Inverse of a 2×2	29.7 μs	5.1 μs	5.8×
Inverse of a 6×6	737 μs	7.0 μs	105×
6×6 · 6 matrix–vector	13.6 μs	1.08 μs	12.6×
6×6 · 6×6 product	77.7 μs	1.32 μs	59×
Inverse of a 60×60	48 μs	66 μs	Mathilda 1.4× ahead

Read the last two rows together. A 60×60 matrix has 100× more elements than a 6×6 and its inverse costs 15× less — because 3600 elements clears `PACK_MIN_ELEMENTS`, so it is packed, so `builtin_inverse` sees an `NDArray`, so it reaches LAPACK. At 36 elements none of that happens and the call runs a fraction-free symbolic Gauss-Jordan with an `Expr` allocation and an `evaluate` per cell per pivot.

Above the threshold Mathilda is already the fastest of the three. Below it, it is 6–105× behind. The threshold is not a performance heuristic here; it is the boundary at which numeric linear algebra exists at all.

The road to fastest

1. Dispatch on "is this a matrix of machine numbers", not "is this already a buffer". `builtin_inverse`, `builtin_dot`, `LinearSolve` and `Det` each test `linalg_call_has_ndarray`; each should instead lift a rectangular all-machine-number argument with `pack_force` regardless of size. Expected: the 6×6 Inverse from 737 μ s to $\sim 2 \mu$ s (LAPACK plus one pack/unpack), the 6×6 product from 77.7 μ s to $\sim 1 \mu$ s. That alone should take the Kalman row from 9.16 s to roughly 0.4 s — level with NumPy.

Do this second, not first. `ndla_inverse` declines by calling `linalg_delist_and_reeval`, which re-evaluates the call; a naive lift re-enters `builtin_inverse`, packs again, declines again, and loops forever — on exactly the singular inputs that are hardest to test. Split each entry point into a `_try` form that returns NULL instead of degrading first.

2. Keep the small buffer between operations. Even with (1), each step packs and unpacks about ten times. A 6×6 that stays an NDArray across the whole `kfstep` pays that once. This falls out of (1) naturally if the numeric paths return packed results, and is worth roughly another 2× on this row.
3. A transposed view (plan 9.1) closes most of the ensemble row: `Transpose[an]` . an on 4096×6 is the dominant term there, and BLAS takes a transpose flag for free.
4. A materialise-free row broadcast. `Table[mn, {4096}]` allocates 4096 copies of a 6-vector to subtract a mean, because `Plus` threads the outer level. A leading-axis broadcast already exists for `Plus/Times` (`packed_broadcast_ok`); the ensemble kernel needs the trailing-axis case.

With 1–4 Mathilda should lead both rows: the arithmetic is trivial and the entire contest is dispatch, which is the one thing a compiled C evaluator ought to win against a Python loop.

Verification

- `Inverse[{{2., 0.}, {0., 4.}}]` now matches Mathematica exactly; the exact (`{{1/2, 0}, {0, 1/4}}`) and symbolic (`{{1/a, 0}, {0, 1/b}}`) cases are unchanged, and `Det`, `LinearSolve` and the 3×3 real inverse were checked alongside.
- Full 395-binary suite clean apart from the one pre-existing `simplify_tests` symbolic-radical printing case confirmed pre-existing in experiments 10 and 11.
- `RowReduce` of a Real matrix still returns exact Integer 1s on the diagonal where Mathematica gives 1. . That is a separate, pre-existing exactness bug in the divide/eliminate steps, already recorded in a comment in `src/linalg/inv.c`; it is not touched here.